TASK 4: Design Classes, Objects and Constructors

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

Classes, fields and methods

Class	Fields	Main methods
Vehicle	regNo (final)	getRegNo(), calculateFee(minutes), equals(), hashCode()
ParkingTicket	ticketNo, vehicle, floor, slot, entryTime	getters for each field
ParkingFacility	slots (jagged array), active (HashMap)	enter(), exit(), release()

Constructors and when each is used

A parameterised constructor is used most, because a vehicle must always have a registration number. A default (no-argument) constructor is not provided on purpose, so that an object without a number cannot be made. Overloaded constructors are used where a second form is useful.

```
class ParkingFacility {
    public ParkingFacility(int[] slotsPerFloor) { ... }   // main constructor
    public ParkingFacility(int floors, int slots) {       // overloaded version
        this(uniform(floors, slots));                     // constructor chaining
    }
}
```

The this keyword and constructor chaining

this(...) calls another constructor of the same class, so checks are written once. this.regNo = regNo separates the field from the parameter with the same name. Example from our code: ParkingTicket's constructor sets all fields once, and a shorter constructor can call it with this(vehicle, floor, slot, currentTime) to avoid repeating the code.

Copy constructor

A copy constructor is not needed. A duplicate ticket should not be a copy of an old ticket: it is a new entry with a new ticket number. Copying would also copy the old ticket number and break the rule that ticket numbers are never reused.

Stack and heap

A variable such as Vehicle v holds a reference (an address) on the stack. The object itself, with its fields, lives on the heap. Two variables can point to one object, so changing the object through one variable is visible through the other. When no variable refers to an object, it becomes eligible for garbage collection.

Alternative: default constructor with setters, or static factory methods

With a default constructor and setters, the object exists before its data is set, so for a short time it can be invalid: a vehicle might have no registration number. Each setter must then check its value, and a missed setter leaves a bad object. A static factory method such as Vehicle.of(type, regNo) validates everything and returns only a valid object, so the object is always valid. Time per object is O(1) in all three designs. Space per object is the same, because the same fields are stored. Creating n tickets costs O(n) in every approach; the factory adds only a constant check per object, while setters add one call per field, which is

slightly slower and gives more chances for error.